

Name:- Kotadiya Vinay Kantibhai | Student ID:- 23101562

GitHub Link:- https://github.com/vinaykotadiya/Image_segmentation_RMDS-

Image Segmentation of Person, Car, Book and Airplane Using ADE20K Dataset

1. Introduction

Semantic segmentation basically assigns a class label to every pixel in an image, which lets you do pretty detailed scene understanding for things like autonomous driving , and also robotics. In this project compares three deep learning models, U-Net, MANet, and DeepLabV3+ , and I segment four object categories (person, car, book, airplane) using a subset of 350 images from the ADE20K dataset.

ADE20K Dataset:

Zhou et al. (2017) introduced the ADE20K dataset, it has more than 20,000 annotated images across 150 object categories. The dataset includes diverse scenarios, plus class imbalance, so it ends up being kind of a common benchmark when you want to judge semantic segmentation models in a fair way.

U-Net and Manet:

Siddique et al. (2021) found that when you add attention mechanisms and multi-scale feature fusion to U-Net, the segmentation quality improves, especially near object edges , and other boundary areas where mistakes are easy. Following that idea, both U-Net and the attention based MANet were tested in this work.

DeepLabV3+:

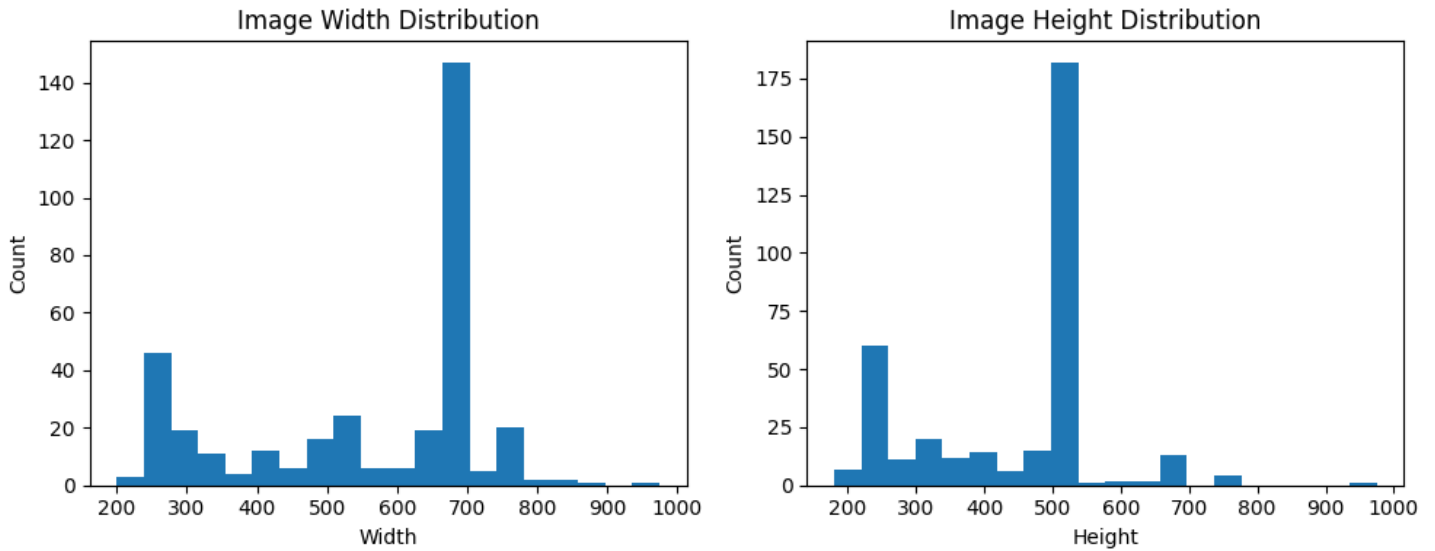
Liu et al. (2024) suggested that DeepLabV3+ benefits from atrous convolutions, and also attention mechanisms, so it can gather multi-scale cues more effectively. As a result, the segmentation performance tends to get better. For that reason, DeepLabV3+ was included as a comparison, to check how well it holds up versus U-Net and MANet, on the chosen ADE20K subset.

2. Data Description and Exploratory Data Analysis

So the dataset ends up having 350 training images and 350 validation ones, with COCO style annotations and 100 categories in total. plus there are 30 unlabelled test images, all of them sizes roughly in the 200 to 975 px range . because of that, every image was resized to 256x256 for the training stage. In the end only 4 of those 100 categories , person car book and airplane, were kept and they became classes 1 through 4. everything else was treated as the background, class 0.

Table 1: Dataset split summary.

Item	Train	Validation	Test
Images	350	350	30
Annotations (all 100 classes)	5,111	5,803	N/A

**Figure 2: Distribution of image widths and heights in the training set.**

The training set has 5,111 annotated instances (1,084 persons, 689 cars, 155 books, just 13 airplanes) which is kind of a strongly long tailed distribution, similar to ADE20K’s typical class frequency. Only 233 / 350 images contain a person , 148 a car, 51 a book, and just 5 an airplane, so the airplane category is rare and kind of bunched up, which in turn limits the accuracy it can realistically reach. Fig. 1 shows a sample image and its generated mask , plus the overlay used for checking the whole mask pipeline.

Table 2: Class instance counts and image coverage in the training set.

Class	Instances (train)	Images containing class
Person	1,084	233
Car	689	148
Book	155	51
Airplane	13	5



Figure 1: Example image, generated multiclass mask, and overlay.

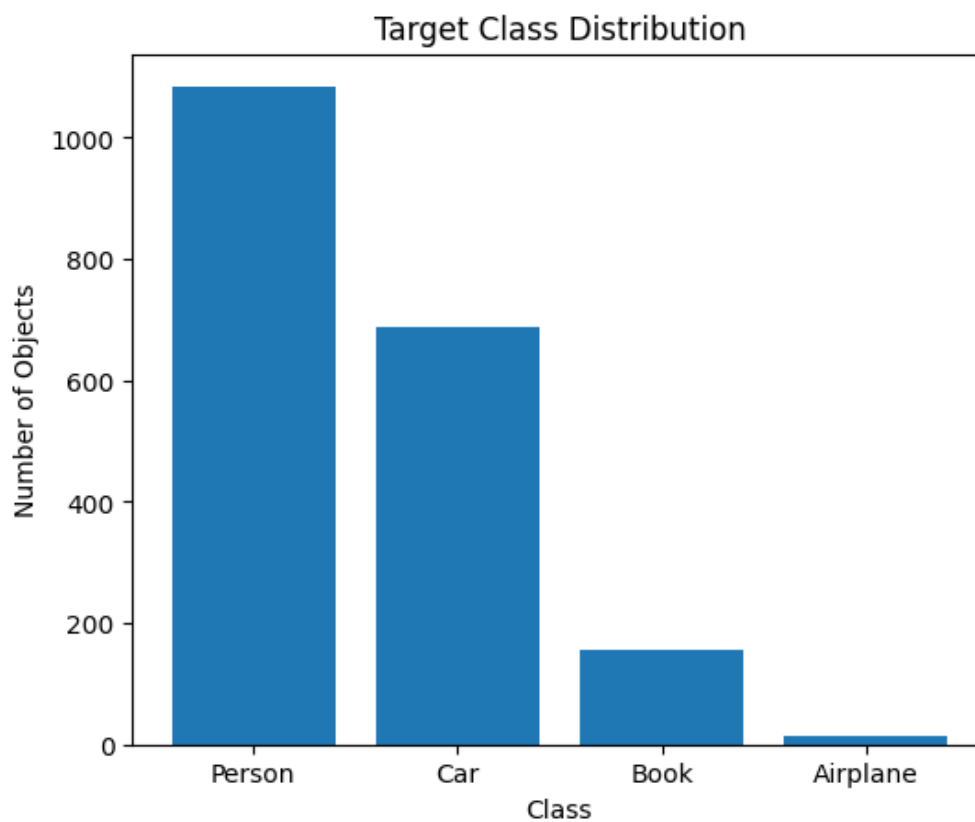


Figure 3: Number of annotated instances per target class.

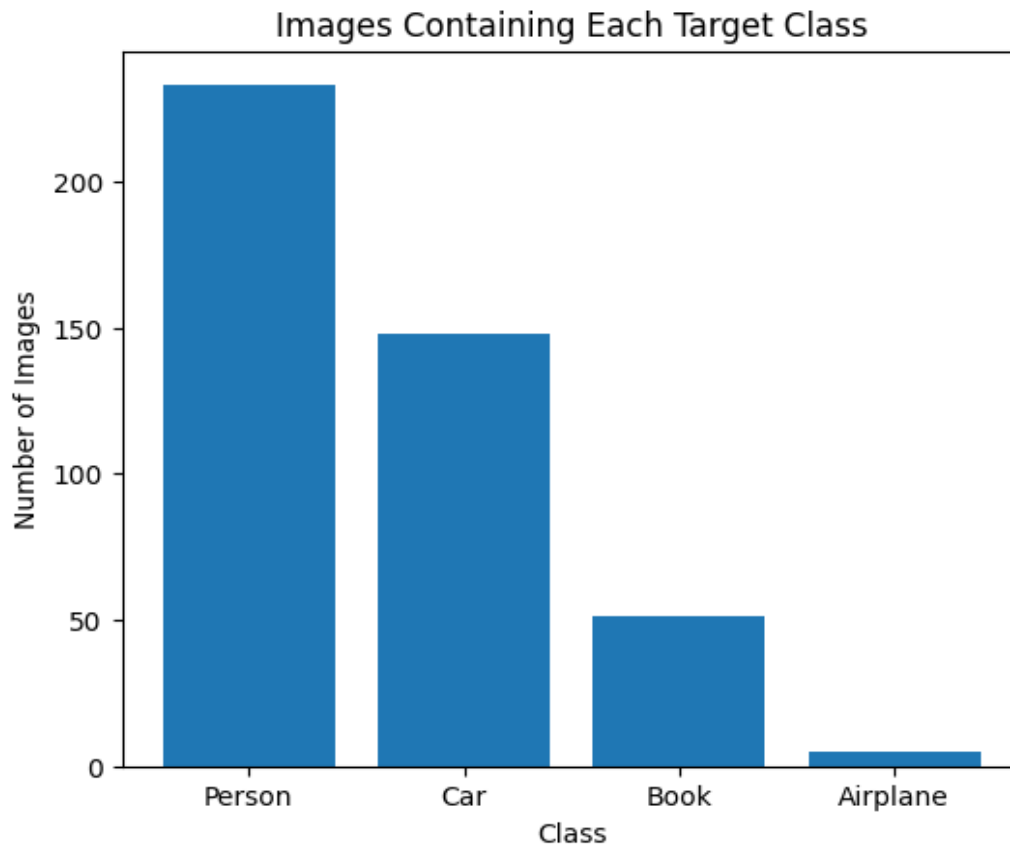


Figure 4: Number of training images containing each target class.

3. Methodology

Instance-level labels were decoded with pycocotools and rasterised into single-channel multiclass masks, with overlapping pixels usually taking the later object’s label, sort of. I then used a custom PyTorch Dataset that does resizing and ImageNet normalisation, and yes during training only it also adds random horizontal flips and small rotations (up to 15°) applied jointly to image and mask. So effectively, each epoch had 44 batches of size 8, no big changes there.

For the models, I trained three setups starting from an ImageNet-pretrained ResNet-34 encoder: first a plain U-Net (about ~24.4M parameters), second MANet, which is an attention-augmented U-Net with multi-scale decoder attention, and third DeepLabV3+ using atrous-convolution in the encoder-decoder. Training used Adam (lr 1e-4, weight decay 1e-5) for 25 epochs, and the loss function was a combined Dice plus cross-entropy loss, intended to offset class imbalance. I also had a ReduceLROnPlateau scheduler (factor 0.5, patience 3), which helps when validation stalls a bit.

For validation we computed mean IoU, pixel accuracy, macro precision/recall, macro F1, and the Dice score, all with torchmetrics. Additionally, I reported per-class IoU specifically for U-Net, as a extra breakdown.

4. Results and Discussion

All three models kinda converged smoothly, training and validation loss going from roughly 1.8-2.5 down to under 0.5 by epoch 25 (Figure 5) , and honestly it looked pretty stable overall. DeepLabV3+ got the better final numbers, with the training loss around 0.30 and validation near 0.44, which hints that its atrous multi-scale context fitting works well even for this smaller dataset.

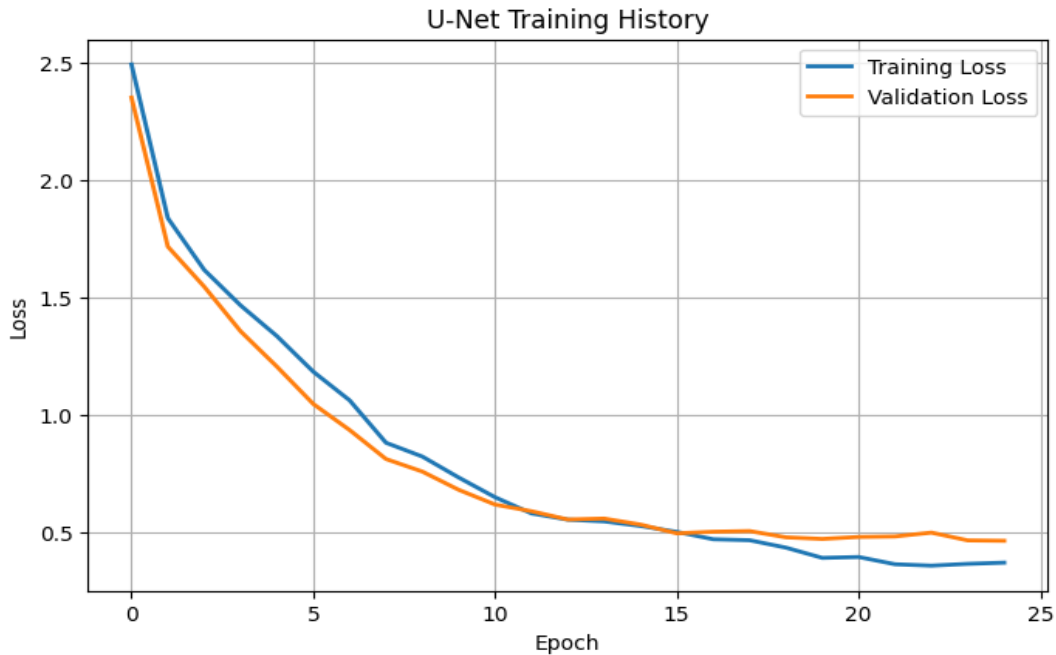


Figure 5a: U-Net training/validation loss.

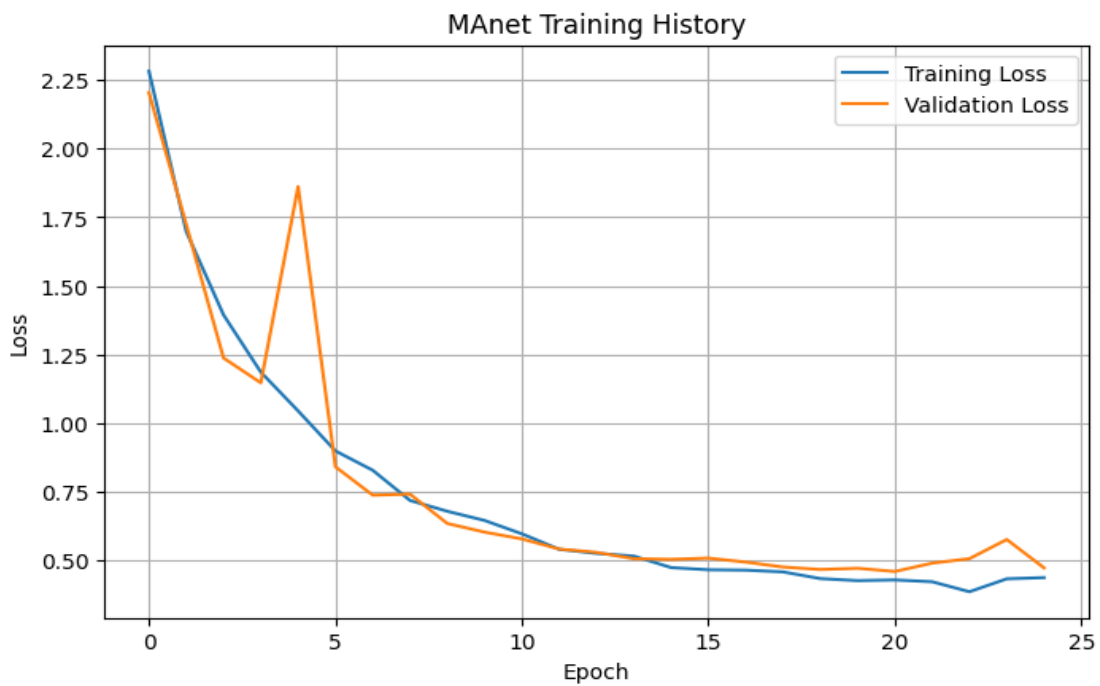


Figure 5b: MAnet training/validation loss.

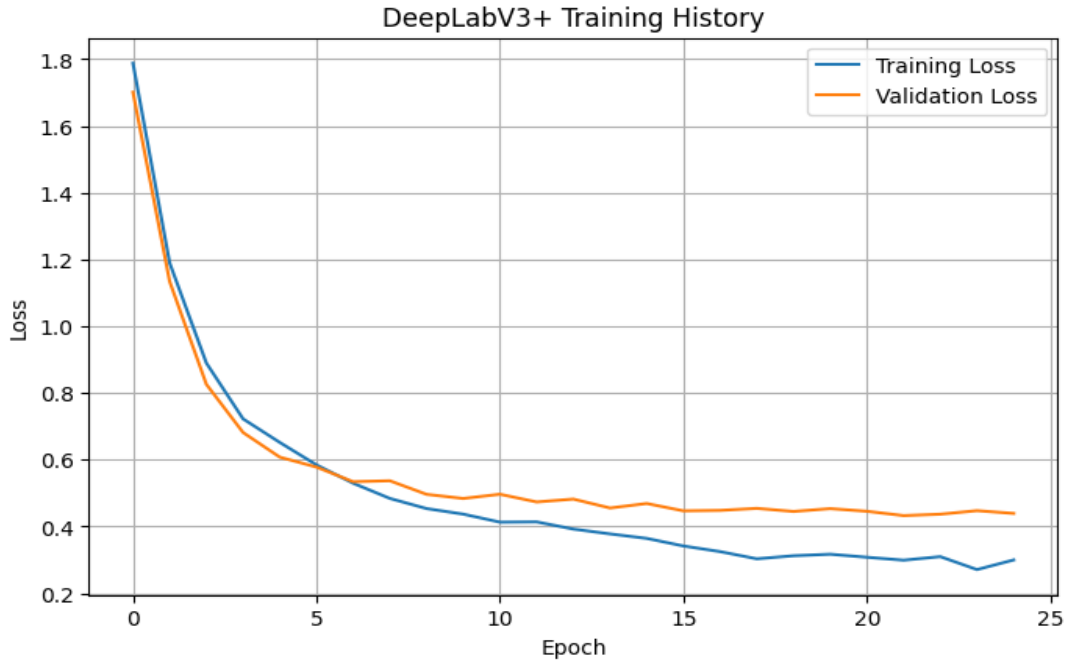


Figure 5c: DeepLabV3+ training/validation loss.

Table 3, shows DeepLabV3+ hitting the best Dice score (0.708) and also precision (0.622) , while U-Net rather clearly leads on mean IoU (0.458) and pixel accuracy (0.562) . MAnet was the weakest across basically every metric. This is a bit non obvious, since Siddique et al. and Liu et al. both say that attention mechanisms improve accuracy, yet their results came from datasets with thousands of images, not this smaller setup. With only 350 training images and a strongly skewed four-class subset, MAnet’s larger attention block parameter total probably made it more susceptible to overfitting and messy optimization dynamics , which shows up as a validation-loss spike around epoch 5 (Figure 5). U-Net on the other hand, being the more straightforward architecture, managed to generalise better from the limited data .

Table 3: Validation metrics for all three models.

Model	Mean IoU	Pixel Acc.	Precision	Recall	F1	Dice
U-Net	0.4579	0.5618	0.5644	0.5618	0.5509	0.7062
MAnet	0.4268	0.4762	0.5106	0.4762	0.4915	0.6962
DeepLabV3+	0.4514	0.5061	0.6222	0.5061	0.5412	0.7077

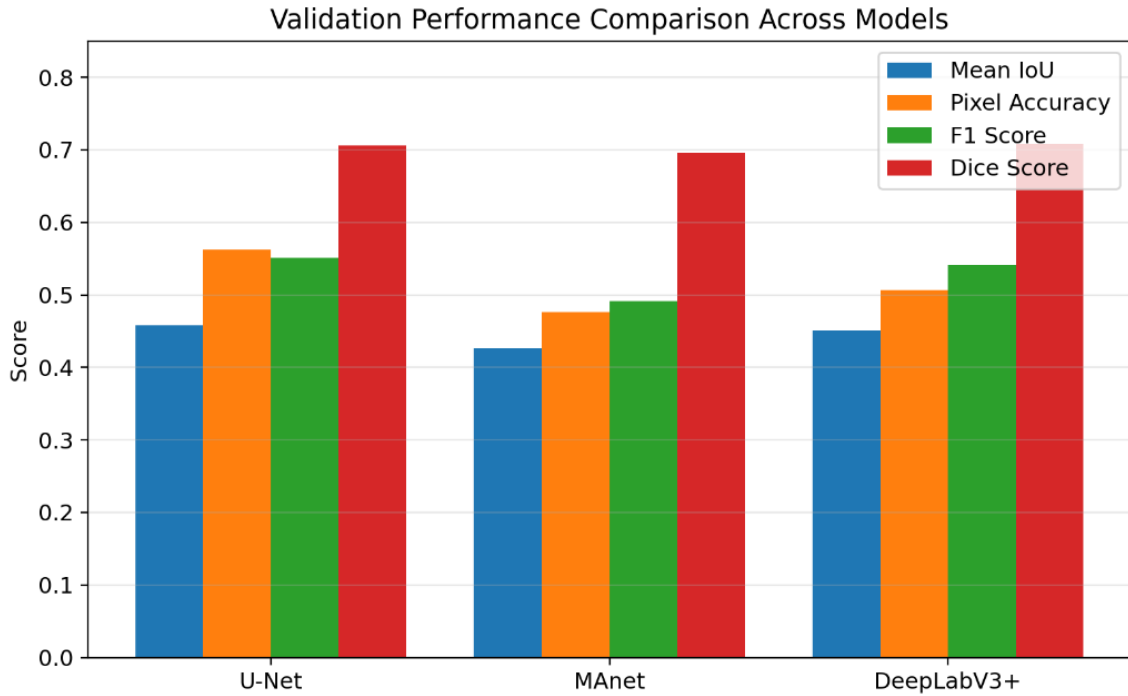


Figure 5d: Validation performance comparison across models (derived from Table 3).

Per-class IoU for U-Net (Table 4 , Figure 6) kind of mirrors the class imbalance we saw in the EDA. Background looks great at 0.953 , person comes to 0.551 and car is 0.592... but book drops to 0.194 and airplane goes down to 0.000. In practice that means the model never predicts airplane correctly, which is pretty much expected since there are only 13 instances total across 5 images. It kinda lines up with the long-tail behavior reported for ADE20K by Zhou et al. and it suggests Dice plus cross-entropy, by themselves , weren't enough to fight that extreme skew. So we likely need a class-weighted loss, or maybe a more targeted oversampling of rare-class images, to get better coverage.

Table 4: U-Net per-class IoU on the validation set.

Class	Background	Person	Car	Book	Airplane
U-Net IoU	0.9532	0.5507	0.5919	0.1937	0.0000

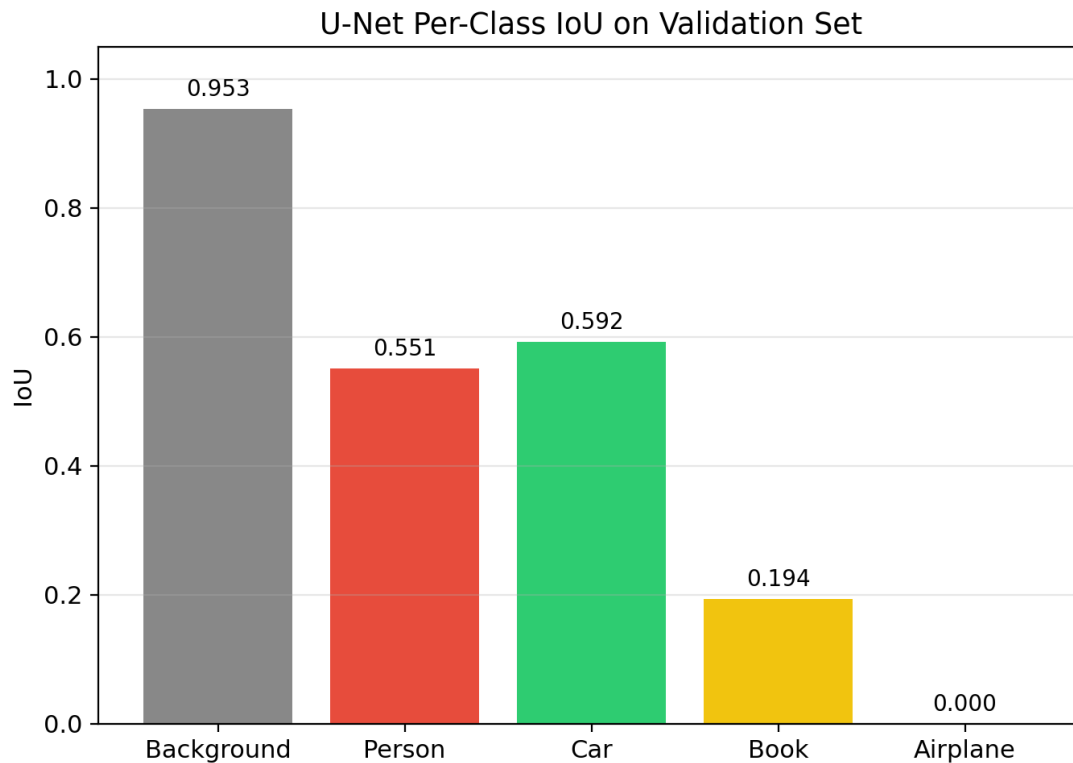


Figure 6: U-Net per-class IoU on the validation set.

On three unseen test images (Figure 7), all three models correctly localise people and cars, with rather coherent boundaries, even if the edges are a bit coarse compared to the ground truth. Also small things or faraway objects are frequently missed too, which is kind of expected. The side by side comparison in Figure 8 kinda confirm that DeepLabV3+ makes the most spatially coherent masks, and it does it with fewer false positives. U-Net, on the other hand, segments person and car outlines slightly more completely, it seems. MAnet masks look visibly noisier again suggesting under training on those rarer classes rather than this being a fundamental architectural issue.

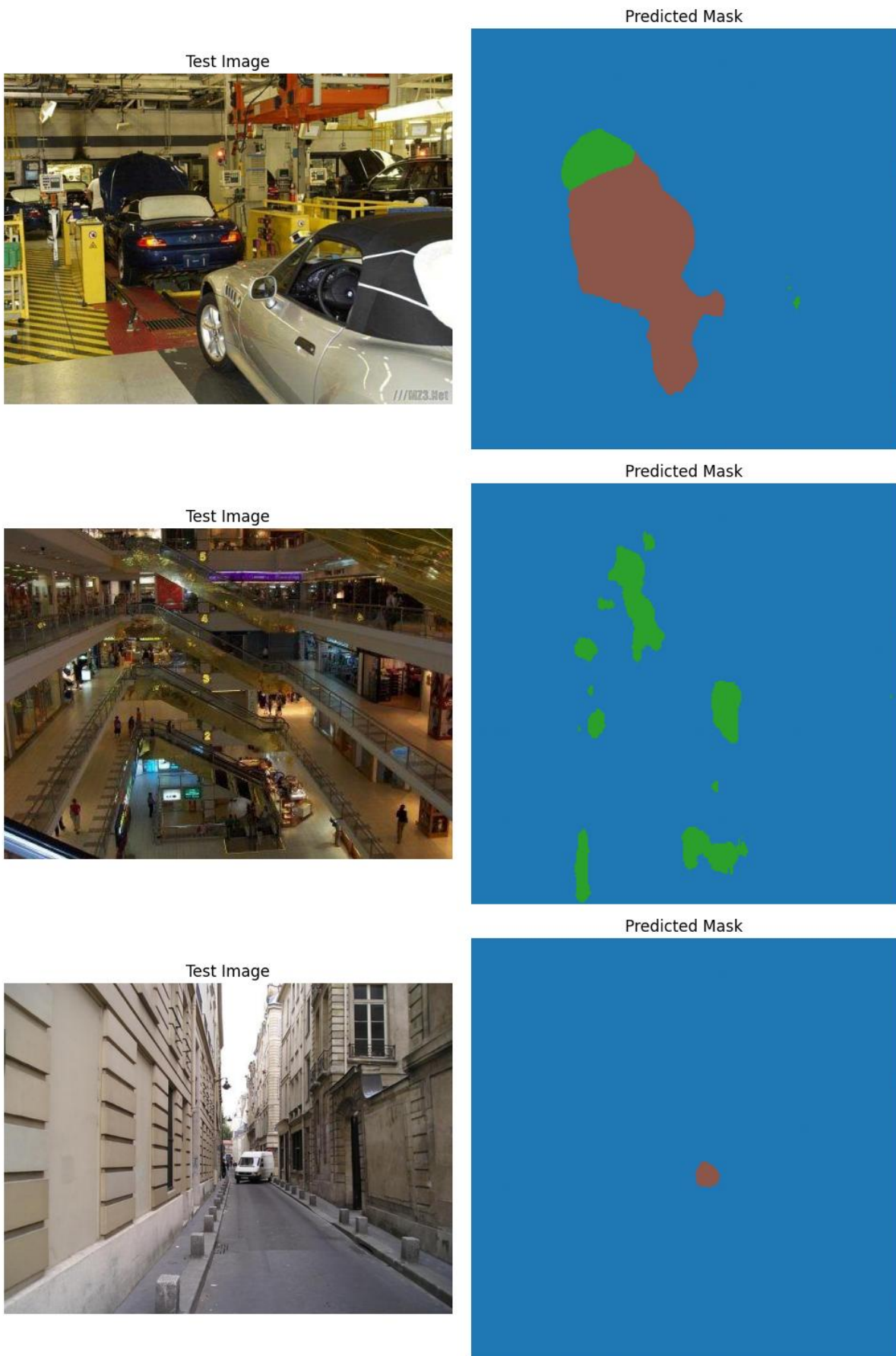


Figure 7: U-Net predictions on three unseen test images.

Model Comparison: Predicted Segmentation Masks

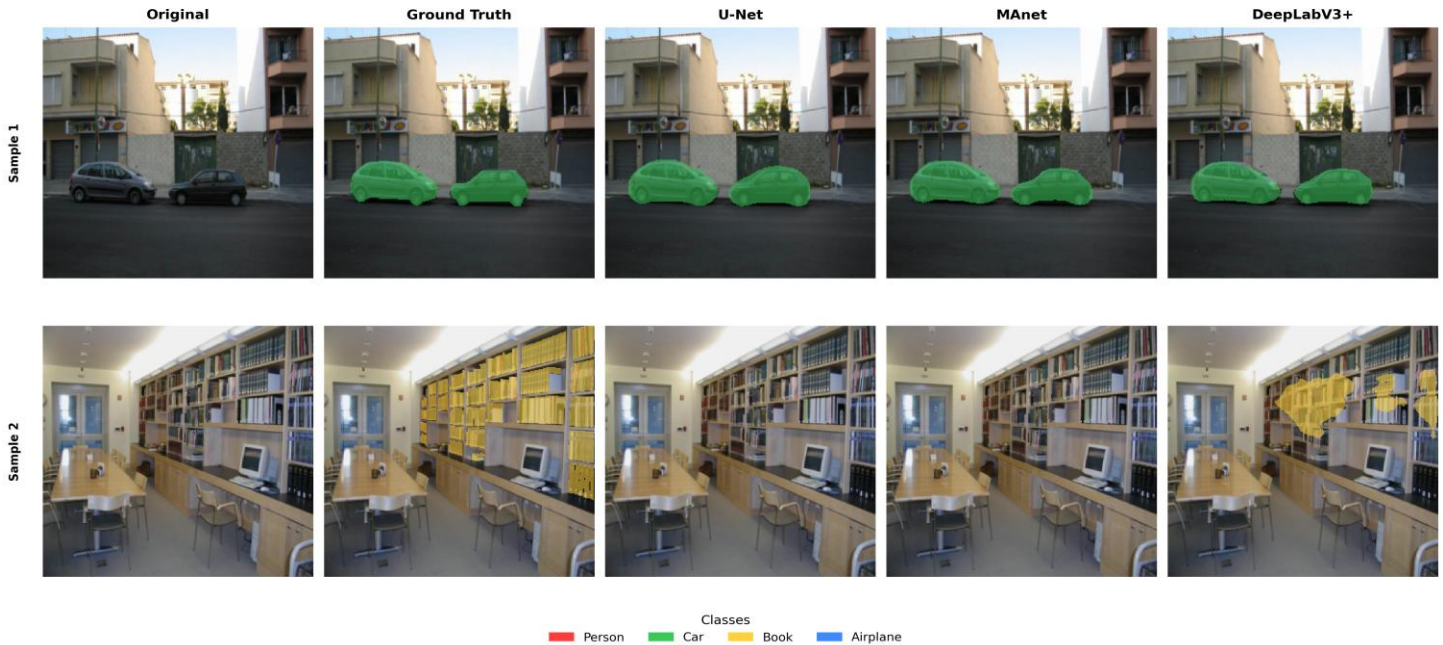


Figure 8: Side-by-side comparison of ground truth and predictions from U-Net, MAnet and DeepLabV3+ on two validation samples.

5. Conclusion

In this setup, three encoder-decoder architectures were trained on a small imbalanced subset of four classes from ADE20K. Kinda surprisingly, DeepLabV3+ came out on top with the best Dice score, plus precision, while U-Net did best for IoU and pixel accuracy. MAnet, on the other hand, kind of fell behind both of them, most plausibly because it overfit the limited and skewed data rather than any real architectural weakness.

Segmentation quality looked decent for person and car, but book not as good, and airplane even worse, mainly because the dataset is too thin there. So, for small specialized segmentation datasets, it seems class-balanced sampling and loss weighting matter just as much, or maybe more than the architecture choice itself.

6. References

1. Liu, Y., Bai, X., Wang, J., Li, G., Li, J. and Lv, Z., 2024. Image semantic segmentation approach based on DeepLabV3 plus network with an attention mechanism. *Engineering Applications of Artificial Intelligence*, 127, p.107260.
(Link: <https://www.sciencedirect.com/science/article/pii/S0952197623014446>)
2. He, P., Jiao, L., Shang, R., Wang, S., Liu, X., Quan, D., Yang, K. and Zhao, D., 2022. MANet: Multi-scale aware-relation network for semantic segmentation in aerial scenes. *IEEE Transactions on Geoscience and Remote Sensing*, 60, pp.1-15.
(Link: <https://ieeexplore.ieee.org/abstract/document/9785828>)
3. Zhou, B., Zhao, H., Puig, X., Fidler, S., Barriuso, A. and Torralba, A., 2017. Scene parsing through ade20k dataset. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 633-641).
(Link: https://openaccess.thecvf.com/content_cvpr_2017/papers/Zhou_Scene_Parsing_Through_CVPR_2017_paper.pdf)
4. Siddique, N., Paheding, S., Elkin, C.P. and Devabhaktuni, V., 2021. U-net and its variants for medical image segmentation: A review of theory and applications. *IEEE access*, 9, pp.82031-82057.
(Link: <https://ieeexplore.ieee.org/abstract/document/9446143>)
5. Alom, M.Z., Yakopcic, C., Hasan, M., Taha, T.M. and Asari, V.K., 2019. Recurrent residual U-Net for medical image segmentation. *Journal of medical imaging*, 6(1), pp.014006-014006.
(Link: <https://pmc.ncbi.nlm.nih.gov/articles/PMC6435980/>)

```
# Setup
```

```
# Mount Google Drive
```

```
from google.colab import drive  
drive.mount('/content/drive')
```

```
# Install segmentation models library if not already installed
```

```
!pip -q install segmentation-models-pytorch  
!pip -q install torchmetrics
```

```
# Import required libraries
```

```
import zipfile  
import os  
import cv2  
import json  
import numpy as np  
from PIL import Image  
from tqdm import tqdm  
import matplotlib.pyplot as plt  
import matplotlib.patches as mpatches  
from pycocotools import mask as maskUtils  
import torch  
from torch.utils.data import Dataset, DataLoader  
from torchvision import transforms  
import segmentation_models_pytorch as smp  
import torch.nn as nn  
import torch.optim as optim  
import torchmetrics  
from torchmetrics.classification import (  
    MulticlassJaccardIndex,  
    MulticlassAccuracy,  
    MulticlassF1Score,
```

```

    MulticlassPrecision,
    MulticlassRecall
)

# Set random seed for reproducibility
import random
import numpy as np
import torch

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)

torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

print("Random seed set to:", SEED)

# Path to the dataset zip file in Google Drive
zip_path = "/content/drive/MyDrive/RMDS_segmentation_dataset_ADE20K_350.zip"

# Folder where the dataset will be extracted
extract_path = "/content/ADE20K_dataset"

# Extract the dataset only if it has not already been extracted

```

```

if not os.path.exists(extract_path):
    with zipfile.ZipFile(zip_path, "r") as zip_ref:
        zip_ref.extractall(extract_path)
    print("Dataset extracted successfully.")
else:
    print("Dataset already extracted.")

# Data Exploration

# Display the extracted folders
print("Contents of extracted dataset:")
print(os.listdir(extract_path))

# Dataset paths
dataset_path = "/content/ADE20K_dataset"

train_dir = os.path.join(dataset_path, "train")
val_dir = os.path.join(dataset_path, "val")
test_dir = os.path.join(dataset_path, "test")

train_json = os.path.join(dataset_path, "instances_train.json")
val_json = os.path.join(dataset_path, "instances_val.json")

# Count images in each folder

train_images = sorted([
    f for f in os.listdir(train_dir)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
])

val_images = sorted([
    f for f in os.listdir(val_dir)

```

```

        if f.lower().endswith((".jpg", ".jpeg", ".png"))
    ])

test_images = sorted([
    f for f in os.listdir(test_dir)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
])

print(f"Training images : {len(train_images)}")
print(f"Validation images : {len(val_images)}")
print(f"Test images : {len(test_images)}")

# Load annotation files

with open(train_json, "r") as f:
    train_data = json.load(f)

with open(val_json, "r") as f:
    val_data = json.load(f)

print("Training JSON loaded.")
print("Validation JSON loaded.")

# Display top-level keys

print("Training JSON keys:")
print(train_data.keys())

# Dataset statistics

print(f"Training images in JSON : {len(train_data['images'])}")
print(f"Training annotations : {len(train_data['annotations'])}")

```

```

print(f"Training categories      : {len(train_data['categories'])}")

# Display all categories

print("Categories:\n")

for category in train_data["categories"]:
    print(f"ID: {category['id']:>3}  Name: {category['name']}")

# Verify the four required classes

required_classes = ["person", "car", "book", "airplane"]

print("Required classes:\n")

for category in train_data["categories"]:
    if category["name"] in required_classes:
        print(category)

# Data Pre-Processing

# Dataset paths
DATASET_PATH = "/content/ADE20K_dataset"

TRAIN_IMAGE_DIR = os.path.join(DATASET_PATH, "train")
VAL_IMAGE_DIR = os.path.join(DATASET_PATH, "val")

TRAIN_JSON = os.path.join(DATASET_PATH, "instances_train.json")
VAL_JSON = os.path.join(DATASET_PATH, "instances_val.json")

# Output folders for generated masks
TRAIN_MASK_DIR = os.path.join(DATASET_PATH, "train_masks")

```



```
VAL_MASK_DIR = os.path.join(DATASET_PATH, "val_masks")
```

```
os.makedirs(TRAIN_MASK_DIR, exist_ok=True)
```

```
os.makedirs(VAL_MASK_DIR, exist_ok=True)
```

```
# Load annotation files
```

```
with open(TRAIN_JSON, "r") as f:
```

```
    train_data = json.load(f)
```

```
with open(VAL_JSON, "r") as f:
```

```
    val_data = json.load(f)
```

```
print("Annotation files loaded successfully.")
```

```
# ADE20K category IDs
```

```
TARGET_CLASSES = {
```

```
    4: 1,  # Person
```

```
    9: 2,  # Car
```

```
    40: 3, # Book
```

```
    59: 4  # Airplane
```

```
}
```

```
CLASS_NAMES = {
```

```
    0: "Background",
```

```
    1: "Person",
```

```
    2: "Car",
```

```
    3: "Book",
```

```
    4: "Airplane"
```

```
}
```

```
print(CLASS_NAMES)
```

```

# Create image lookup dictionary
train_images = {
    image["id"]: image
    for image in train_data["images"]
}

val_images = {
    image["id"]: image
    for image in val_data["images"]
}

print(f"Training image records : {len(train_images)}")
print(f"Validation image records : {len(val_images)}")

# Install COCO API
!pip -q install pycocotools

# Output folders for segmentation masks
TRAIN_MASK_DIR = os.path.join(DATASET_PATH, "train_masks")
VAL_MASK_DIR = os.path.join(DATASET_PATH, "val_masks")

os.makedirs(TRAIN_MASK_DIR, exist_ok=True)
os.makedirs(VAL_MASK_DIR, exist_ok=True)

print("Output folders created.")

# Target classes
TARGET_CLASSES = {
    4: 1,  # Person
    9: 2,  # Car
    40: 3, # Book
    59: 4  # Airplane

```

```
}
```

```
CLASS_NAMES = {  
    0: "Background",  
    1: "Person",  
    2: "Car",  
    3: "Book",  
    4: "Airplane"  
}
```

```
print(CLASS_NAMES)
```

```
# Generate multiclass segmentation masks
```

```
def generate_masks(annotation_data, image_lookup, output_dir):
```

```
    # Group annotations by image
```

```
    annotations_by_image = {}
```

```
    for ann in annotation_data["annotations"]:
```

```
        annotations_by_image.setdefault(ann["image_id"], []).append(ann)
```

```
    # Create one mask for each image
```

```
    for image_id, image_info in tqdm(image_lookup.items(), desc="Generating Masks"):
```

```
        height = image_info["height"]
```

```
        width = image_info["width"]
```

```
        mask = np.zeros((height, width), dtype=np.uint8)
```

```
        annotations = annotations_by_image.get(image_id, [])
```

```
        for ann in annotations:
```

```

category = ann["category_id"]

if category not in TARGET_CLASSES:
    continue

class_id = TARGET_CLASSES[category]

binary_mask = maskUtils.decode(ann["segmentation"])

mask[binary_mask == 1] = class_id

mask_name = os.path.splitext(image_info["file_name"])[0] + ".png"

Image.fromarray(mask).save(
    os.path.join(output_dir, mask_name)
)

print(f"Saved masks to {output_dir}")

generate_masks(
    train_data,
    train_images,
    TRAIN_MASK_DIR
)

generate_masks(
    val_data,
    val_images,
    VAL_MASK_DIR
)

```

```

# Verify one generated mask
import random
import matplotlib.pyplot as plt

image_info = random.choice(list(train_images.values()))

image = Image.open(
    os.path.join(TRAIN_IMAGE_DIR, image_info["file_name"])
)

mask = Image.open(
    os.path.join(
        TRAIN_MASK_DIR,
        os.path.splitext(image_info["file_name"])[0] + ".png"
    )
)

print("Unique mask values:", np.unique(mask))

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(mask, cmap="tab10", vmin=0, vmax=4)
plt.title("Generated Mask")
plt.axis("off")

plt.show()

```

```

# Check the classes present in the displayed mask
mask_array = np.array(mask)

print("Unique mask values:", np.unique(mask_array))

# EDA

# Display dataset summary
print("ADE20K Dataset Summary")
print()

print(f"Training Images    : {len(train_images)}")
print(f"Validation Images   : {len(val_images)}")
print(f"Test Images          : {len(test_images)}")

print(f"\nTraining Annotations : {len(train_data['annotations'])}")
print(f"Validation Annotations: {len(val_data['annotations'])}")

print(f"\nTotal Categories    : {len(train_data['categories'])}")

print("\nTarget Classes")
for category_id, class_id in TARGET_CLASSES.items():
    print(f"{CLASS_NAMES[class_id]} (ADE20K ID: {category_id})")

# Analyse image dimensions
image_widths = []
image_heights = []

for image in train_data["images"]:
    image_widths.append(image["width"])
    image_heights.append(image["height"])

```

```
print(f"Minimum Width : {min(image_widths)}")
print(f"Maximum Width : {max(image_widths)}")
```

```
print(f"Minimum Height : {min(image_heights)}")
print(f"Maximum Height : {max(image_heights)}")
```

```
plt.figure(figsize=(10,4))
```

```
plt.subplot(1,2,1)
plt.hist(image_widths, bins=20)
plt.title("Image Width Distribution")
plt.xlabel("Width")
plt.ylabel("Count")
```

```
plt.subplot(1,2,2)
plt.hist(image_heights, bins=20)
plt.title("Image Height Distribution")
plt.xlabel("Height")
plt.ylabel("Count")
```

```
plt.tight_layout()
plt.show()
```

```
# Count object instances
```

```
class_counts = {
    "Person": 0,
    "Car": 0,
    "Book": 0,
    "Airplane": 0
}
```

```

for ann in train_data["annotations"]:

    if ann["category_id"] == 4:
        class_counts["Person"] += 1

    elif ann["category_id"] == 9:
        class_counts["Car"] += 1

    elif ann["category_id"] == 40:
        class_counts["Book"] += 1

    elif ann["category_id"] == 59:
        class_counts["Airplane"] += 1

print(class_counts)

plt.figure(figsize=(6,5))

plt.bar(class_counts.keys(), class_counts.values())

plt.title("Target Class Distribution")
plt.xlabel("Class")
plt.ylabel("Number of Objects")

plt.show()

plt.figure(figsize=(6,6))

plt.pie(
    class_counts.values(),
    labels=class_counts.keys(),
    autopct="%1.1f%%",

```



```

    startangle=90
)

plt.title("Target Class Percentage")

plt.show()

# Count how many images contain each target class
images_per_class = {
    "Person": set(),
    "Car": set(),
    "Book": set(),
    "Airplane": set()
}

category_mapping = {
    4: "Person",
    9: "Car",
    40: "Book",
    59: "Airplane"
}

for ann in train_data["annotations"]:

    if ann["category_id"] in category_mapping:
        class_name = category_mapping[ann["category_id"]]
        images_per_class[class_name].add(ann["image_id"])

image_counts = {
    key: len(value)
    for key, value in images_per_class.items()
}

```

```

print(image_counts)

plt.figure(figsize=(6,5))

plt.bar(image_counts.keys(), image_counts.values())

plt.title("Images Containing Each Target Class")
plt.xlabel("Class")
plt.ylabel("Number of Images")

plt.show()

# Display one image with its segmentation overlay
image_info = random.choice(list(train_images.values()))

image = np.array(
    Image.open(
        os.path.join(TRAIN_IMAGE_DIR, image_info["file_name"])
    )
)

mask = np.array(
    Image.open(
        os.path.join(
            TRAIN_MASK_DIR,
            os.path.splitext(image_info["file_name"])[0] + ".png"
        )
    )
)

plt.figure(figsize=(15,5))

```

```

plt.subplot(1,3,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(mask, cmap="tab10", vmin=0, vmax=4)
plt.title("Segmentation Mask")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(image)
plt.imshow(mask, cmap="jet", alpha=0.45, vmin=0, vmax=4)
plt.title("Mask Overlay")
plt.axis("off")

plt.tight_layout()
plt.show()

# DataLoader

# Training configuration
IMAGE_SIZE = 256
BATCH_SIZE = 8
NUM_WORKERS = 2

print(f"Image Size : {IMAGE_SIZE}")
print(f"Batch Size : {BATCH_SIZE}")

# Image transformations
IMAGE_SIZE = 256

```

```

image_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

```

```

mask_transform = transforms.Compose([
    transforms.Resize(
        (IMAGE_SIZE, IMAGE_SIZE),
        interpolation=Image.NEAREST
    )
])

```

Dataset class

```

import torchvision.transforms.functional as TF
import random

```

```

class ADE20KDataset(Dataset):

```

```

    def __init__(self, image_dir, mask_dir, transform=None, mask_transform=None, augment=False):

```

```

        self.image_dir = image_dir
        self.mask_dir = mask_dir
        self.transform = transform
        self.mask_transform = mask_transform
        self.augment = augment

```

```

        self.images = sorted([

```

```

        file for file in os.listdir(image_dir)
        if file.endswith(".jpg")
    ])

def __len__(self):
    return len(self.images)

def __getitem__(self, index):

    image_name = self.images[index]

    image_path = os.path.join(self.image_dir, image_name)

    mask_name = image_name.replace(".jpg", ".png")

    mask_path = os.path.join(self.mask_dir, mask_name)

    image = Image.open(image_path).convert("RGB")
    mask = Image.open(mask_path)

    if self.transform:
        image = self.transform(image)

    if self.mask_transform:
        mask = self.mask_transform(mask)

    mask = np.array(mask, dtype=np.int64)

    mask = torch.tensor(mask, dtype=torch.long)

    if self.augment:
        if random.random() > 0.5:

```

```
image = TF.hflip(image)
mask = TF.hflip(mask.unsqueeze(0)).squeeze(0)
```

```
if random.random() > 0.5:
    angle = random.uniform(-15, 15)
    image = TF.rotate(image, angle)
    mask = TF.rotate(
        mask.unsqueeze(0), angle,
        interpolation=transforms.InterpolationMode.NEAREST
    ).squeeze(0)
```

```
return image, mask
```

```
# Create datasets
```

```
train_dataset = ADE20KDataset(
    TRAIN_IMAGE_DIR,
    TRAIN_MASK_DIR,
    image_transform,
    mask_transform,
    augment=True
)
```

```
val_dataset = ADE20KDataset(
    VAL_IMAGE_DIR,
    VAL_MASK_DIR,
    image_transform,
    mask_transform,
    augment=False
)
```

```
print("Training samples :", len(train_dataset))
print("Validation samples :", len(val_dataset))
```

```

# Create DataLoaders
train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=torch.cuda.is_available()
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=torch.cuda.is_available()
)

print("Train batches :", len(train_loader))
print("Validation batches :", len(val_loader))

# Verify one batch
images, masks = next(iter(train_loader))

print("Image batch shape :", images.shape)
print("Mask batch shape :", masks.shape)

print("Mask labels :", torch.unique(masks))

# Check all generated masks
all_labels = set()

```

```

for mask_file in os.listdir(TRAIN_MASK_DIR):

    mask = np.array(
        Image.open(
            os.path.join(TRAIN_MASK_DIR, mask_file)
        )
    )

    all_labels.update(np.unique(mask))

print("Labels found in training masks:", sorted(all_labels))

# Verify one batch
images, masks = next(iter(train_loader))

print("Image batch shape :", images.shape)
print("Mask batch shape :", masks.shape)

print("Mask labels :", torch.unique(masks))

# U-Net Model

# Create U-Net model
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model_unet = smp.Unet(
    encoder_name="resnet34",
    encoder_weights="imagenet",
    in_channels=3,
    classes=5,
    activation=None
)

```



```

model_unet = model_unet.to(device)

print("U-Net created successfully.")

# Define loss functions
dice_loss = smp.losses.DiceLoss(mode="multiclass")
cross_entropy_loss = nn.CrossEntropyLoss()

def segmentation_loss(predictions, targets):
    return (
        dice_loss(predictions, targets)
        + cross_entropy_loss(predictions, targets)
    )

print("Combined loss function created.")

# Define optimizer
optimizer = optim.Adam(
    model_unet.parameters(),
    lr=1e-4,
    weight_decay=1e-5
)

print("Optimizer created.")

# Define scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,

```

```

    patience=3
)

print("Scheduler created.")

# Count trainable parameters
trainable_parameters = sum(
    parameter.numel()
    for parameter in model_unet.parameters()
    if parameter.requires_grad
)

print(f"Trainable Parameters: {trainable_parameters:,}")

# Training configuration
EPOCHS = 25

print(f"Epochs: {EPOCHS}")
print(f"Batch Size: {BATCH_SIZE}")
print(f"Image Size: {IMAGE_SIZE}")
print(f"Device: {device}")

# Train for one epoch
def train_one_epoch(model, dataloader, optimizer):

    model.train()

    running_loss = 0.0

    for images, masks in dataloader:

        images = images.to(device)

```

```

    masks = masks.to(device)

    optimizer.zero_grad()

    outputs = model(images)

    loss = segmentation_loss(outputs, masks)

    loss.backward()

    optimizer.step()

    running_loss += loss.item()

return running_loss / len(dataloader)

# Validate for one epoch
def validate_one_epoch(model, dataloader):

    model.eval()

    running_loss = 0.0

    with torch.no_grad():

        for images, masks in dataloader:

            images = images.to(device)
            masks = masks.to(device)

            outputs = model(images)

```

```

    loss = segmentation_loss(outputs, masks)

    running_loss += loss.item()

return running_loss / len(dataloader)

# Train model
def train_model(model, train_loader, val_loader, optimizer, scheduler, epochs, checkpoint_path):

    train_losses = []
    val_losses = []

    best_val_loss = float("inf")

    for epoch in range(epochs):

        train_loss = train_one_epoch(
            model,
            train_loader,
            optimizer
        )

        val_loss = validate_one_epoch(
            model,
            val_loader
        )

        scheduler.step(val_loss)

        train_losses.append(train_loss)
        val_losses.append(val_loss)

```

```

if val_loss < best_val_loss:

    best_val_loss = val_loss

    torch.save(
        model.state_dict(),
        checkpoint_path
    )

    print(
        f"Epoch {epoch + 1}/{epochs} | "
        f"Train Loss: {train_loss:.4f} | "
        f"Validation Loss: {val_loss:.4f}"
    )

print(f"Training completed. Best model saved to {checkpoint_path}")

return train_losses, val_losses

# Train U-Net
train_losses, val_losses = train_model(
    model=model_unet,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer=optimizer,
    scheduler=scheduler,
    epochs=EPOCHS,
    checkpoint_path="best_unet.pth"
)

# Plot training history
import matplotlib.pyplot as plt

```

```

plt.figure(figsize=(8, 5))

plt.plot(train_losses, label="Training Loss", linewidth=2)
plt.plot(val_losses, label="Validation Loss", linewidth=2)

plt.title("U-Net Training History")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()

# Install evaluation library
!pip -q install torchmetrics

NUM_CLASSES = 5

iou_metric = MulticlassJaccardIndex(
    num_classes=NUM_CLASSES,
    average="macro"
).to(device)

accuracy_metric = MulticlassAccuracy(
    num_classes=NUM_CLASSES,
    average="macro"
).to(device)

precision_metric = MulticlassPrecision(
    num_classes=NUM_CLASSES,
    average="macro"

```

```
).to(device)
```

```
recall_metric = MulticlassRecall(  
    num_classes=NUM_CLASSES,  
    average="macro"  
).to(device)
```

```
f1_metric = MulticlassF1Score(  
    num_classes=NUM_CLASSES,  
    average="macro"  
).to(device)
```

```
# Evaluate model  
model_unet.load_state_dict(torch.load("best_unet.pth"))  
model_unet.eval()
```

```
iou_metric.reset()  
accuracy_metric.reset()  
precision_metric.reset()  
recall_metric.reset()  
f1_metric.reset()
```

```
with torch.no_grad():
```

```
    for images, masks in val_loader:
```

```
        images = images.to(device)  
        masks = masks.to(device)
```

```
        outputs = model_unet(images)
```

```
        predictions = torch.argmax(outputs, dim=1)
```

```

iou_metric.update(predictions, masks)
accuracy_metric.update(predictions, masks)
precision_metric.update(predictions, masks)
recall_metric.update(predictions, masks)
f1_metric.update(predictions, masks)

print(f"Mean IoU      : {iou_metric.compute():.4f}")
print(f"Pixel Accuracy : {accuracy_metric.compute():.4f}")
print(f"Precision      : {precision_metric.compute():.4f}")
print(f"Recall         : {recall_metric.compute():.4f}")
print(f"F1 Score       : {f1_metric.compute():.4f}")

# Dice Score
dice_metric = smp.losses.DiceLoss(mode="multiclass")

model_unet.eval()

dice_scores = []

with torch.no_grad():

    for images, masks in val_loader:

        images = images.to(device)
        masks = masks.to(device)

        outputs = model_unet(images)

        loss = dice_metric(outputs, masks)

        dice_scores.append(1 - loss.item())

```



```

print(f"Dice Score: {np.mean(dice_scores):.4f}")

# Per-class IoU
from torchmetrics.classification import MulticlassJaccardIndex

class_names = [
    "Background",
    "Person",
    "Car",
    "Book",
    "Airplane"
]

per_class_iou = MulticlassJaccardIndex(
    num_classes=5,
    average=None
).to(device)

per_class_iou.reset()

model_unet.eval()

with torch.no_grad():

    for images, masks in val_loader:

        images = images.to(device)
        masks = masks.to(device)

        outputs = model_unet(images)

```

```

predictions = torch.argmax(outputs, dim=1)

per_class_iou.update(predictions, masks)

scores = per_class_iou.compute().cpu().numpy()

for name, score in zip(class_names, scores):
    print(f"{name:12s}: {score:.4f}")

# Show predictions on validation images
model_unet.eval()

images, masks = next(iter(val_loader))

images = images.to(device)

with torch.no_grad():
    outputs = model_unet(images)

predictions = torch.argmax(outputs, dim=1).cpu()

images = images.cpu()

num_samples = 3

fig, axes = plt.subplots(num_samples, 3, figsize=(12, 12))

for i in range(num_samples):

    image = images[i].permute(1, 2, 0).numpy()

    image = image * np.array([0.229, 0.224, 0.225]) + np.array([0.485, 0.456, 0.406])

```

```

image = np.clip(image, 0, 1)

axes[i, 0].imshow(image)
axes[i, 0].set_title("Original")
axes[i, 0].axis("off")

axes[i, 1].imshow(masks[i], cmap="tab10", vmin=0, vmax=4)
axes[i, 1].set_title("Ground Truth")
axes[i, 1].axis("off")

axes[i, 2].imshow(predictions[i], cmap="tab10", vmin=0, vmax=4)
axes[i, 2].set_title("Prediction")
axes[i, 2].axis("off")

plt.tight_layout()
plt.show()

# Predict on unseen test images
import random

model_unet.eval()

test_images = sorted([
    file for file in os.listdir(test_dir)
    if file.endswith(".jpg")
])

samples = random.sample(test_images, 3)

fig, axes = plt.subplots(3, 2, figsize=(10, 15))

for i, image_name in enumerate(samples):

```

```

image_path = os.path.join(test_dir, image_name)

image = Image.open(image_path).convert("RGB")

input_image = image_transform(image).unsqueeze(0).to(device)

with torch.no_grad():
    prediction = model_unet(input_image)

prediction = torch.argmax(prediction, dim=1).squeeze().cpu().numpy()

axes[i, 0].imshow(image)
axes[i, 0].set_title("Test Image")
axes[i, 0].axis("off")

axes[i, 1].imshow(prediction, cmap="tab10", vmin=0, vmax=4)
axes[i, 1].set_title("Predicted Mask")
axes[i, 1].axis("off")

plt.tight_layout()
plt.show()

# Attention U-Net

print(smp.__version__)
print(hasattr(smp, "MANet"))
print(hasattr(smp, "Unet"))

# Create MANet model
model_manet = smp.MANet(
    encoder_name="resnet34",

```

```

    encoder_weights="imagenet",
    in_channels=3,
    classes=5,
    activation=None
)

model_manet = model_manet.to(device)

print("MANet created successfully.")

# Optimizer
optimizer_manet = optim.Adam(
    model_manet.parameters(),
    lr=1e-4,
    weight_decay=1e-5
)

print("Optimizer created.")

# Scheduler
scheduler_manet = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer_manet,
    mode="min",
    factor=0.5,
    patience=3
)

print("Scheduler created.")

# Train MANet
train_losses_manet, val_losses_manet = train_model(
    model=model_manet,

```

```
train_loader=train_loader,  
val_loader=val_loader,  
optimizer=optimizer_manet,  
scheduler=scheduler_manet,  
epochs=EPOCHS,  
checkpoint_path="best_manet.pth"  
)
```

```
# Plot MANet training history
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(train_losses_manet, label="Training Loss")
```

```
plt.plot(val_losses_manet, label="Validation Loss")
```

```
plt.title("MANet Training History")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
# Evaluate segmentation model
```

```
def evaluate_model(model, dataloader):
```

```
    model.eval()
```

```
    iou_metric.reset()
```

```
    accuracy_metric.reset()
```

```
    precision_metric.reset()
```

```
    recall_metric.reset()
```

```
    f1_metric.reset()
```

```

dice_scores = []

with torch.no_grad():

    for images, masks in dataloader:

        images = images.to(device)
        masks = masks.to(device)

        outputs = model(images)

        predictions = torch.argmax(outputs, dim=1)

        iou_metric.update(predictions, masks)
        accuracy_metric.update(predictions, masks)
        precision_metric.update(predictions, masks)
        recall_metric.update(predictions, masks)
        f1_metric.update(predictions, masks)

        dice_scores.append(
            1 - dice_loss(outputs, masks).item()
        )

print(f"Mean IoU      : {iou_metric.compute():.4f}")
print(f"Pixel Accuracy : {accuracy_metric.compute():.4f}")
print(f"Precision      : {precision_metric.compute():.4f}")
print(f"Recall         : {recall_metric.compute():.4f}")
print(f"F1 Score       : {f1_metric.compute():.4f}")
print(f"Dice Score     : {np.mean(dice_scores):.4f}")

model_manet.load_state_dict(torch.load("best_manet.pth"))

```

```
evaluate_model(model_manet, val_loader)
```

```
# DeepLabV3+
```

```
# Create DeepLabV3+ model
```

```
model_deeplab = smp.DeepLabV3Plus(  
    encoder_name="resnet34",  
    encoder_weights="imagenet",  
    in_channels=3,  
    classes=5,  
    activation=None  
)
```

```
model_deeplab = model_deeplab.to(device)
```

```
print("DeepLabV3+ created successfully.")
```

```
# Optimizer
```

```
optimizer_deeplab = optim.Adam(  
    model_deeplab.parameters(),  
    lr=1e-4,  
    weight_decay=1e-5  
)
```

```
print("Optimizer created.")
```

```
# Scheduler
```

```
scheduler_deeplab = optim.lr_scheduler.ReduceLROnPlateau(  
    optimizer_deeplab,  
    mode="min",  
    factor=0.5,  
    patience=3  
)
```



```
)
```

```
print("Scheduler created.")
```

```
# Train DeepLabV3+
```

```
train_losses_deeplab, val_losses_deeplab = train_model(
```

```
    model=model_deeplab,
```

```
    train_loader=train_loader,
```

```
    val_loader=val_loader,
```

```
    optimizer=optimizer_deeplab,
```

```
    scheduler=scheduler_deeplab,
```

```
    epochs=EPOCHS,
```

```
    checkpoint_path="best_deeplab.pth"
```

```
)
```

```
# Plot DeepLabV3+ training history
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(train_losses_deeplab, label="Training Loss")
```

```
plt.plot(val_losses_deeplab, label="Validation Loss")
```

```
plt.title("DeepLabV3+ Training History")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
model_deeplab.load_state_dict(torch.load("best_deeplab.pth"))
```

```
evaluate_model(model_deeplab, val_loader)
```

Model Comparison

```
image_ids = [159,289]
colors = {1: [255,60,60], 2: [60,200,90], 3: [255,210,60], 4: [60,140,255]}
labels = {1: "Person", 2: "Car", 3: "Book", 4: "Airplane"}
models = {"U-Net": model_unet, "MAnet": model_manet, "DeepLabV3+": model_deeplab}

def overlay(img, mask, alpha=0.55):
    out = img.copy()
    for cls, c in colors.items():
        m = mask == cls
        if m.any():
            out[m] = out[m]*(1-alpha) + np.array(c)/255*alpha
    return out

fig, axes = plt.subplots(len(image_ids), 5, figsize=(22, 5.5*len(image_ids)), dpi=220, facecolor="white")
if len(image_ids) == 1:
    axes = np.expand_dims(axes, 0)

titles = ["Original", "Ground Truth"] + list(models.keys())

for row, idx in enumerate(image_ids):
    image, mask = val_dataset[idx]
    img = image.permute(1,2,0).numpy() * [0.229,0.224,0.225] + [0.485,0.456,0.406]
    img = np.clip(img, 0, 1)
    gt = mask.numpy()

    panels = [img, overlay(img, gt)]
    for model in models.values():
        with torch.no_grad():
            pred = model(image.unsqueeze(0).to(device))
```

```

pred = torch.argmax(pred, dim=1).squeeze().cpu().numpy()
panels.append(overlay(img, pred))

for col, panel in enumerate(panels):
    axes[row, col].imshow(panel)
    axes[row, col].axis("off")
    if row == 0:
        axes[row, col].set_title(titles[col], fontsize=15, fontweight="bold", pad=10)

axes[row, 0].text(-0.08, 0.5, f"Sample {row+1}", transform=axes[row,0].transAxes,
                  fontsize=13, fontweight="bold", va="center", ha="right", rotation=90)

fig.legend(handles=[mpatches.Patch(color=np.array(c)/255, label=labels[cls]) for cls, c in colors.items()],
           title="Classes", loc="lower center", ncol=4, fontsize=13, title_fontsize=14,
           frameon=False, bbox_to_anchor=(0.5, -0.02))

fig.suptitle("Model Comparison: Predicted Segmentation Masks", fontsize=18, fontweight="bold", y=1.02)
plt.subplots_adjust(left=0.04, right=0.98, top=0.90, bottom=0.08, wspace=0.04, hspace=0.12)
plt.show()

```